PAGE NO.:
DATE: / /

1. Describe the Agile Software Development process and the Agile Manifesto.

→ Agile Software Development.

Agile Software Development is a software development method that focuses on flexibility, teamwork, customer feedback, and continuous improvement. Instead of developing the complete software at once, the software is developed and delivered in small parts called increments.

Agile Development process

The main steps are:

1. Requirement Gathering : Collect and prioritize customer requirements.

2. Planning : Plan the work for a short development cycle called a sprint.

3. Design : Prepare a simple and flexible design

4. Development : Develop the required features step by step.

5. Testing : Test the software continuously during development.

6. Deployment : Deliver the working software to users.

7. Review and Feedback : Take customer feedback and plan improvements.

This process is repeated for every iteration until the complete product is developed.

PAGE NO.:
DATE: / /

Agile Manifesto:

The Agile Manifesto was created in 2001 by software experts at snowbird. It provides the basic values of Agile development.

The four values are:

1. Individuals and interactions over process and tools.

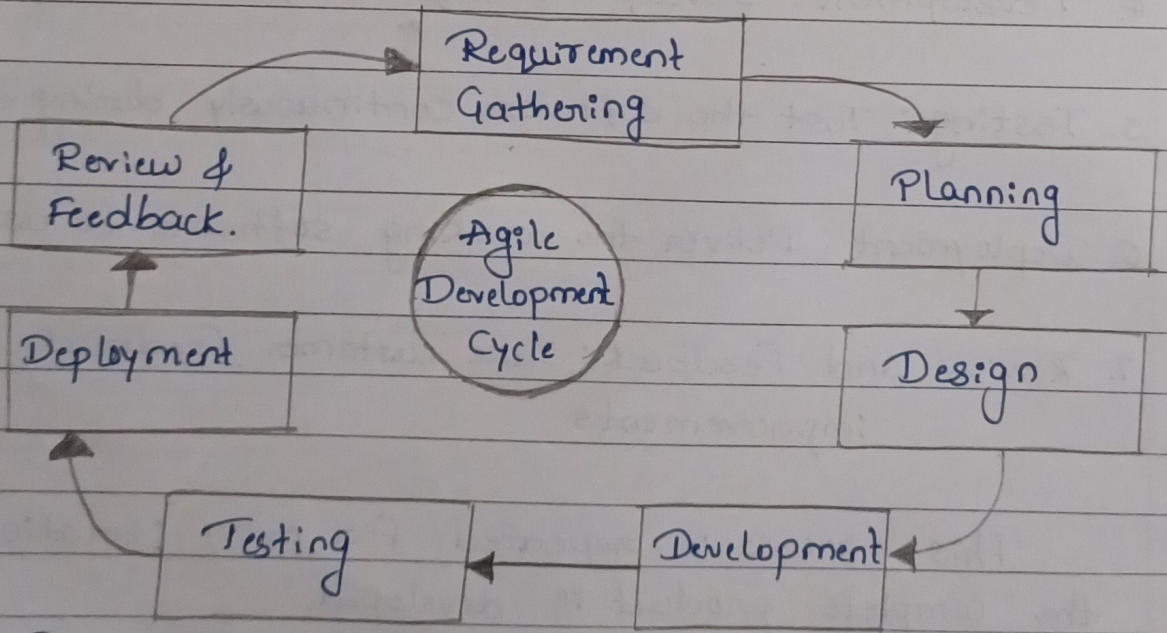
2. Working software over comprehensive documentation.

3. Customer collaboration over contract negotiation.

4. Responding to change over following a plan.

2. Illustrate the Agile Design and Development process with a suitable diagram.

Agile design and development is an itterative process. Design, development, testing and feedback are repeated in small cycle. The system can be improved whenever new requirements or feedback are received.



Repeat cycle for continuous improvement & better result

PAGE NO.:
DATE: / /

Explanation:

1. Requirements : Customer requirements are collected.

2. Planning : Tasks are selected for the current iteration.

3. Design : A simple design is prepared

4. Development : Features are developed

5. Testing : The developed features are tested.

6. Working Software : A usable part of the software is produced.

7. Deployment : The software is delivered.

8. Feedback : Customer feedback is collected.

9. Next Iteration : changes and improvements are included in the next cycle.

Important Agile design practices include simple design, incremental design, refactoring, user-story-based design, and collaborative design.

PAGE NO
DATE / /

03. Analyze the significance of the 12 Agile Principles in Software development.

→ Agile Principles in Software development.

1. Customer Satisfaction :
Deliver useful software early & continuously to keep the customer satisfied.

2. Welcome changing Requirements:
Changes in requirements should be accepted, even during development.

3. Frequent delivery:
Deliver working software regularly in small releases.

4. Customer and developer cooperation :
Customer and developers should work together throughout the project

5. Motivated team members :
Give team members the support and environment they need to do their work well.

6. Direct Communication :
Face-to-face communication is an effective way to share information.

7. Working Software is important :
The main measure of progress is working software

8. Maintain a steady development pace :
The team should work at a pace that can be maintained for a long time.

9. Good technical quality:
Good design, coding, and technical practices improve the software.

10. Keep things simple:
Avoid unnecessary work and unneccessary complexity.

11. Self - organizing teams:
Teams should organize their own work and make suitable decisions.

12. Regular improvement:
Team should regularly check their work and improvement their process.

Significance:
These principles help Agile teams to:

- Deliver software faster.
- Handle changing requirements.
- Improve software quality
- Increase customer satisfaction
- Improve teamwork and communication
- Reduce project risk
- Continuosly improve the development process.

PAGE NO.:
DATE: / /

4. Examine the key concepts of Lean Software Development.

→ Lean Software Development is an Agile method that focuses on removing waste, improving efficiency, delivering customer value, and continuous improvement. It is derived from Lean manufacturing principles developed in the Toyota production system.

Seven Principles of Lean Software Development.

1. Eliminate Waste:

Remove activities that do not provide value to customer.

Examples:
- Unnecessary documentation
- Extra features.
- Rework
- Delays
- Unused code.

2. Build Quality In:

Quality should be maintained throughout development instead of checking it only at the end.

Examples:
- Continuous testing
- code reviews
- Refactoring.

3. Create Knowledge:

Teams should continuously learn through discussions, feedback, and experience.

4. Defer Commitment:

Important decisions should be made at the right time when enough information is available.

5. Deliver Fast:

Deliver software quickly in small releases.

6. Respect People:

Encourage teamwork, communication, and motivation

PAGE NO.:
DATE: / /

7. Optimize the Whole:
Improve the complete development process instead of improving only one part.

Advantages:
- Reduces development cost.
- Faster software development.
- Improve Software quality.
- Increase customer satisfaction.
- Improve team productivity.

5. Demonstrate the workflow of Kanban in Agile Software Development with an example.

-> Kanban is an Agile method used to visualize work, manage workflow, improve productivity, and deliver software continuously. It uses a visual board to show the status of tasks.

Kanban Workflow.

Test - Backlog
↓
To Do
↓
In Progress
↓
Testing
↓
Done.

Tasks move from left to right as the work progresses.

Example: Suppose a team is developing an online shopping app.

To Do	InProgress	Testing	Done
Search Product	Payment	Login	Registration
Wishlist			

PAGE NO.:
DATE: / /

- To Do : Tasks that have not started.
- In progress : Tasks currently being developed.
- Testing : Completed development tasks are being teste
- Done : Successfully completed tasks.

Main Kanban Principles :

1. Visualize Work - Show all tasks on the board.
2. Limit Work in progress - Do not work on too many task once.
3. Manage Flow : Make sure tasks move smoothly
4. Make Rules clear : Define how tasks move between stage
5. Use Feedback : Take regular feedback
6. Improve Continuously : Regularly improve the workflow.

Advantages :

- Easy to Understand.
- Flexible.
- Improves workflow visibility.
- Reduces bottlenecks.
- Supports continuous delivery
- Improves productivity.

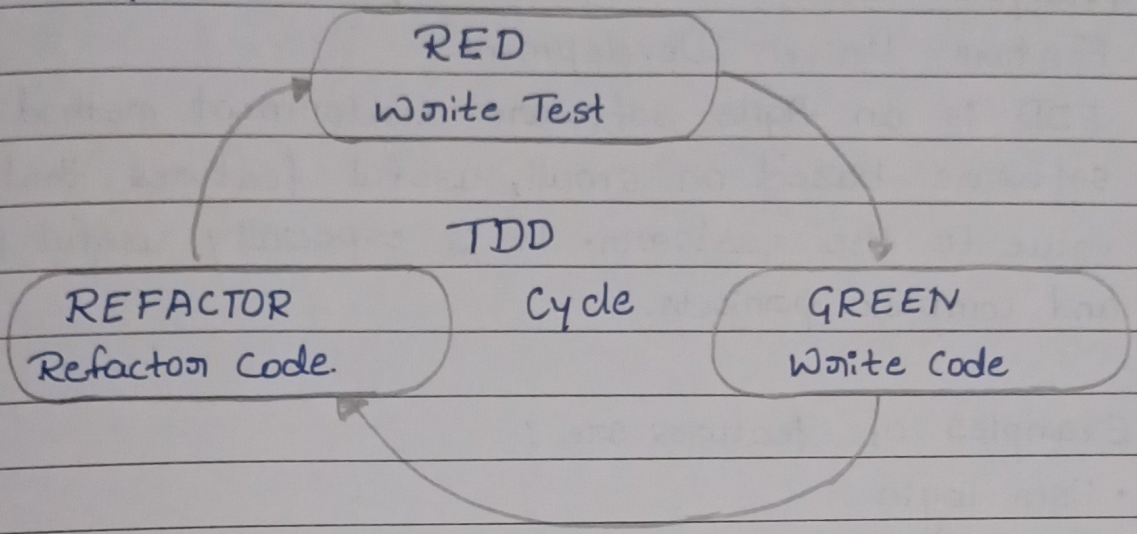
Describe Test-Driven Development and its advantages.

Test - Driven Development [TDD]

TDD is an Agile development practice in which test cases are written before writing the actual program code.

PAGE NO.:
DATE: / /

TDD cycle :



Three Phases :

1. Red - Write a test for a new feature. The test fails because the features is not yet implemented.

2. Green - Write the minimum code required to make the test pass.

3. Refactor - Improve the code without changing its functionality.

Advantages of TDD.

- Improves Software quality.
- Finds bugs early.
- Produces cleaner code.
- Makes maintenance easier.
- Gives quick feedback
- Increases developer confidence
- Supports continuous improvement.

PAGE NO.:
DATE: / /

7. Analyze Feature-Driven Development (FDD)

Feature-Driven Development.

FDD is an Agile software development method that develops software based on small, useful features that provide value to the customer. It is especially useful for large and complex projects.

Examples of features are:

- User login
- Product Search
- Generate Report
- Online Payment.

Five Main Process of FDD

1. Develop an Overall Model

Create a high-level model of the complete system.

2. Build a feature List.

Prepare a list of small and useful features.

3. Plan by Feature.

Prioritize features and assign them to developers.

4. Design by Feature.

Create the detailed design for selected features.

5. Build by Feature.

Code, test and integrate the features into the system.

PAGE NO.:
DATE: / /

Advantages:
- Suitable for large projects.
- Easy to track progress.
- Provides frequent software delivery.
- Improves code quality
- Improves team coordination
- Supports scalable development.

Limitations:
- Requires experienced developers.
- Less suitable for small projects
- Depends strongly on good modeling
- Customer involvement may be limited.

8. Illustrate the practices of Extreme Programming [XP] with suitable examples.

Extreme Programming (XP)

Extreme programming is an Agile methodology that focuses on high software quality, customer involvement, teamwork, frequent releases, and quick response to changes. It was developed by Kent Beck in the late 1990s.

Key Practices of XP.

1. Pair Programming:

Two developers work together on one computer.
- One developer writes the code
- The other reviews the code.

2. Test-Driven Development [TDD]:

Tests are written before the actual code

3. Continuous Integration:

Developers regularly combine their code and test it.

4. Refactoring:
Improve the structure of code without changing its functionality.

5. Small Releases:
Releases software in small and frequent versions.

6. Simple Design:
Keep the design simple and easy to understand.

7. Customer Involvement:
Customers actively participate and provide feedback

8. Collective Code Ownership:
Any developer can modify any part of the code.

Examples:

Suppose a team is developing an online shopping application:

- Customer gives requirements.
- Developers write tests before coding.
- Two developers use pair programming
- Code is integrated every day.
- New features are released every two weeks.
- Customer checks the release and gives feedback.
- The team uses the feedback in the next iteration.

Advantages of XP.

- High-quality software
- Quick response to changes.
- Early detection of defects.
- Better customer satisfaction.